

Linguagem C++ - Coleções: Containers Sequenciais - Arranjos

Leonardo Medeiros

2025

Objetivos de Aprendizado

- Depois dessa unidade você deve ser capaz de:
 - Declarar, acessar elementos, inicializar e usar arranjos em C++
 - Usar `std::string` e `std::string_view` para tarefas que envolvem texto

Tópicos

- Containers Sequenciais
 - Arranjos Estilo C
 - `std::array`
 - `std::vector`
 - Strings Estilo C
 - `std::string`
 - `std::string_view`

Problema de Armazenar Muitos Valores

- Exemplo: Um professor que quer calcular a média de 30 estudantes.
- No programa precisamos de 30 variáveis para armazenar as 30 médias, e ainda 30 vezes a quantidade de notas que compõem a média.
- Detalhe: cada variável deverá ter um nome diferente!

Containers Sequenciais

- **Containers** são tipos de dados que fornecem armazenamento para uma **coleção** de objetos (elementos).
- Containers Sequenciais armazenam elementos de forma **linear** na memória e fornecem acesso rápido e direto para qualquer elemento.
- Os containers sequenciais do C++ são:
 - Array (arranjo fixo)
 - Vector (arranjo dinâmico)
 - String (arranjo dinâmico de caracteres)
 - Deque (fila dupla)
 - List (lista encadeada)
 - Forward List (lista progressiva)

Arranjos Estilo C

- Herança da linguagem C, foi incluído no C++ para possuir retrocompatibilidade.
- **Arranjo agrupa dados do mesmo tipo** e possibilita trabalhar com grande volume de dados.
 - Exemplo: Caixa de ferramentas (6), Caixa de ovos (2x3, 2x6, 4x5, 6x5)

Arranjos Estilo C

- Arranjos de uma dimensão são chamados **vetores**, e de duas ou mais dimensões de **matrizes**.
- Usamos arranjos pois facilitam o gerenciamento de muitos elementos.
- Acesso aos elementos através de **índices** (uma dimensão = um índice, duas dimensões = dois índices, ...).

Vetores

- Características:
 - Todos os elementos são do mesmo tipo.
 - Arranjos de uma única **dimensão**.
 - Necessita de apenas um **índice** para acessar seus elementos.

Sintaxe de Vetores

```
<tipo> <nome>[tamanho da dimensão];
```

Exemplos de Declaração de Vetores

```
int idades[50];  
char nomes[200];  
float precos[30];
```

Acesso a Elementos dos Vetores

- Com o vetor declarado, poderemos armazenar e ler os valores dos seus elementos.
- O acesso aos elementos é através do nome do vetor seguido do índice entre colchetes (operador []).
- Os índices dos vetores na linguagem C/C++ iniciam com o índice 0.

Índice	0	1	2	...
Valor	<valor1>	<valor2>	<valor3>	...

Exemplo de Acesso a Elementos dos Vetores

- Exemplo 10-01: Programa em C++ que declara e acessa elementos de um vetor de inteiros.

Exemplo de Acesso a Elementos dos Vetores: ex10-01.cpp

```
#include <iostream>
using namespace std;
int main(){
    int numeros[3] = { 3, 42, 73 }; //Inicialização do vetor
    cout << numeros[2] << '\n'; //Exibe o terceiro valor do vetor
    cin >> numeros[2]; //Armazena valor no vetor
    cout << numeros[2] << '\n'; //Exibe o terceiro valor do vetor
    return 0;
}
```

Exemplo de Vetores

- Exemplo 10-02: Programa em C++ que preenche por leitura um vetor de 10 elementos com valores reais que representam temperaturas, exibe todas as temperaturas, determina e exibe a menor temperatura.

Exemplo de Vetores: ex10-02.cpp

```
#include <iostream>
using namespace std;
int main(){
    float t[10];
    float menor;
    for (int i = 0; i <= 9; i++) {
        cin >> t[i];
        if (i == 0)
            menor = t[0];
        if (t[i] < menor)
            menor = t[i];
    }
    for (int i = 0; i <= 9; i++)
        cout << t[i] << " ";
    cout << "\nMenor: " << menor << '\n';
    return 0;
}
```

Matrizes

- Características:
 - Todos os elementos são do mesmo tipo.
 - Arranjos de duas ou mais **dimensões** (multidimensionais).
 - Necessita de um **índice** para cada dimensão.
 - Matrizes bidimensionais necessitam de 2 índices;
 - Matrizes tridimensionais de 3 índices e assim sucessivamente.

Sintaxe de Matrizes

```
<tipo> <nome>[dimensão 1][dimensão 2] ... [dimensão n];
```

Exemplos de Declaração de Matrizes

```
int idades[50][5];  
char nomes[200][5];  
float precos[30][5][10];
```

Acesso a Elementos dos Matrizes

- Com a matriz declarada, poderemos armazenar e ler os valores dos seus elementos.
- O acesso aos elementos é através do nome do vetor seguido dos índices entre colchetes (Operador []).
- Os índices das matrizes na linguagem C/C++ iniciam com o índice 0.

Índices	0	1	2	...
0	<valor[0][0]>	<valor[0][1]>	<valor[0][2]>	...
1	<valor[1][0]>	<valor[1][1]>	<valor[1][2]>	...
2	<valor[2][0]>	<valor[2][1]>	<valor[2][2]>	...

Exemplo de Acesso a Elementos dos Matrizes

- Exemplo 10-04: Programa em C que declara e acessa elementos de uma matriz de inteiros com 2 linhas e 3 colunas.

Exemplo de Acesso a Elementos dos Matrizes: ex10-03.cpp

```
#include <iostream>
using namespace std;
int main(){
    int numeros[2][3] = {
        { 3, 42, 73},
        {4, 43, 74}
    };
    cout << numeros[1][1] << '\n';
    cin >> numeros[1][1];
    cout << numeros[1][1] << '\n';
    return 0;
}
```

Exemplo de Matrizes

- Exemplo 10-04: Programa em C que preenche uma matriz bidimensional de 4 linhas e 3 colunas com números inteiros e exibe a soma dos valores da última coluna.

Exemplo de Matrizes: ex10-04.cpp

```
#include <iostream>
using namespace std;
int main(){
    int n[4][3];
    int soma = 0;
    int i, j;
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 3; j++) {
            cin >> n[i][j];
            if (j == 2)
                soma += n[i][2];
        }
    }
    cout << "Soma: " << soma;
    return 0;
}
```

Container `std::array`

- Arranjo fixo, requer que o tamanho seja conhecido na instanciação e não pode ser alterado
- `std::array` é definido no cabeçalho `<array>`
- Foi introduzido no C++11 como substituto direto dos arranjos estilo C.

```
#include <array>
```


Sintaxe do `std::array`

```
std::array<tipo,tamanho> nome {};
```

Exemplo de Declaração do std::array

- tamanho precisa ser uma expressão constante (constante literal, variável constexpr ou enum), evite macros.

```
std::array<int, 7> a {}; // Constante literal
constexpr int tam { 8 };
std::array<int, len> b {}; // Variável constexpr
enum Cores {vermelho, verde, azul, max_cores};
std::array<int, max_cores> c {}; // Enumerador
```

Inicialização Agregada do `std::array`

- `std::array` é um agregado, não possui construtor e é inicializado usando inicialização agregada.

- Exemplo:

```
std::array<int, 6> fibonnaci = { 0, 1, 1, 2, 3, 5 };  
std::array<int, 5> primos { 2, 3, 5, 7, 11 };           // preferencial
```

- Não-inicializado

```
std::array<int, 5> a;    // Elementos com inicialização padrão  
                        // (elementos int são deixados sem inicialização)  
std::array<int, 5> b {}; // Elementos com valores inicializados  
                        // (elementos int são inicializados com zero) (preferencial)
```

std::array const ou constexpr

- Um std::array pode ser const ou constexpr

```
const std::array<int, 5> primos { 2, 3, 5, 7, 11 };  
constexpr std::array<int, 5> primos { 2, 3, 5, 7, 11 };
```

- Defina std::array como constexpr sempre que possível, senão considere o uso do std::vector

Dedução dos Argumentos do `std::array`

- A partir do C++17 o compilador pode deduzir o tipo e o tamanho.

```
constexpr std::array a1 { 9, 7, 5, 3, 1 }; // Deduzido como std::array<int, 5>
constexpr std::array a2 { 9.7, 7.31 };    // Deduzido como std::array<double, 2>
```

Acesso aos Elementos do `std::array` com o Operador `[ ]`

- O modo mais comum de acessar os elementos de um `std::array` é usando o operador `[ ]`

```
constexpr std::array<int, 5> primos { 2, 3, 5, 7, 11 };  
std::cout << primos[3]; // imprime o valor do elemento com índice 3 (7)  
std::cout << primos[9]; // índice inválido (comportamento indefinido)
```

- O operador `[ ]` não inclui verificação dos limites dos índices

Acesso aos Elementos do `std::array` com Verificação dos Limites dos Índices

- Podemos usar `std::get` para acessar elementos verificando os limites dos índices

```
constexpr std::array primos { 2, 3, 5, 7, 11 };  
std::cout << std::get<3>(primos); // imprime o valor do elemento com índice 3  
std::cout << std::get<9>(primos); // índice inválido (erro de compilação)
```

Comprimento do `std::array`

- Existem três formas de recuperar o Comprimento de um `std::array`

```
constexpr std::array vetor { 9, 7, 5, 3, 1 };  
std::cout << "Comprimento: " << vetor.size() << '\n';  
// retorna comprimento do tipo `size_type` (atalho para `std::size_t`)  
std::cout << "Comprimento: " << std::size(vetor);  
// C++17, retorna comprimento do tipo `size_type` (atalho para `std::size_t`)  
std::cout << "Comprimento: " << std::ssize(vetor);  
// C++20, retorna comprimento do tipo long signed int
```


Container `std::vector`

- Arranjo dinâmico
- `std::vector` é definido no cabeçalho `<vector>`
- Foi introduzido no C++03 para tornar os arranjos mais seguros e fáceis.
- É o mais flexível, e possui capacidades que os outros arranjos não possuem.

```
#include <vector>
```

Sintaxe do `std::vector`

```
std::vector<tipo> nome {};
```

Exemplo de Declaração do `std::vector`

```
std::vector<int> vazio {}; // vector contendo 0 elementos int
```

Construtor de Lista

- Containers possuem tipicamente um construtor especial chamado **construtor de lista**
- Permite construir uma instância do container usando uma lista de inicialização
- O construtor de lista:
 - Garante que o container terá espaço suficiente para armazenar todos os valores da inicialização (se necessário)
 - Determina o tamanho do container para o número de elementos na lista de inicialização (se necessário)
 - Inicializa os elementos com os valores da lista de inicialização (em ordem sequencial)

Inicialização do `std::vector` com uma Lista de Valores

- `std::vector` é inicializado usando uma lista de inicialização.
- Exemplo:

```
// Construtor de Lista
std::vector<int> primos { 2, 3, 5, 7 };
// vector contendo 4 elementos int com valores 2, 3, 5, e 7
std::vector<char> vogais { 'a', 'e', 'i', 'o', 'u' };
// vector contendo 5 elementos char com valores 'a', 'e', 'i', 'o', e 'u'.
//Usa CTAD (C++17) para deduzir tipo char dos elementos (preferencial).
```

Acesso aos Elementos do `std::vector` com o Operador []

- O modo mais comum de acessar os elementos de um `std::vector` é usando o operador []

```
std::vector<int> primos { 2, 3, 5, 7, 11 }; // hold the first 5 prime numbers (as int)
std::cout << "O primeiro número primo é: " << primos[0] << '\n';
std::cout << "O segundo número primo é: " << primos[1] << '\n';
std::cout << "A soma dos 5 primeiros primos é: " <<
primos[0] + primos[1] + primos[2] + primos[3] + primos[4] << '\n';
```

- O operador [] não inclui verificação dos limites dos índices

Construindo `std::vector` de Tamanho Específico

- Pode ser definido através de uma lista com a quantidade de valores desejada
- Existe um construtor específico que permite definir o tamanho

```
std::vector<int> dados { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };  
// vector contendo 10 valores int  
std::vector<int> dados ( 10 );  
// vector contendo 10 elementos int, valores inicializados com 0
```

Interpretação da Inicialização do `std::vector`

- Para ajudar a clarificar os vários casos de inicialização

```
// Inicialização por cópia
```

```
std::vector<int> v1 = 10;
```

```
// 10 não é uma lista de inicialização, não corresponde a um construtor explícito:
```

```
// Inicialização Direta
```

```
std::vector<int> v2 (10);
```

```
// 10 não é uma lista de inicialização, corresponde a um construtor explícito com a
```

```
// Lista de Inicialização
```

```
std::vector<int> v3 { 10 };
```

```
// { 10 } interpretado como lista de inicialização, corresponde a um construtor de
```

```
// Inicialização por cópia de lista
```

```
std::vector<int> v4 = { 10 };
```

```
// { 10 } interpretado como lista de inicialização, corresponde a um construtor de
```

```
std::vector<int> v5 ({ 10 });
```

```
// { 10 } interpretado como lista de inicialização, corresponde a um construtor de
```


`std::vector` `const` e não `constexpr`

- Um `std::vector` pode ser `const`, mas não pode ser `constexpr`

```
const std::vector<int> primos { 2, 3, 5, 7, 11 };  
// primos e seus elementos não podem ser modificados
```

- Se precisar de um arranjo `constexpr` use `std::array`

Strings Estilo C

- Literais strings são fáceis de utilizar.

```
#include <iostream>
int main(){
    std::cout << "Hello, world!";
    return 0;
}
```

- Porém variáveis strings tem seus incômodos:
 - Não dá pra usar atribuição
 - Ter cuidado com espaço atribuído para evitar comportamento indefinido.

Tipos String no C++

- O C++ introduziu dois tipos de string adicionais que são muito mais fáceis e seguros de se trabalhar:
 - `std::string`
 - `std::string_view` (C++17)
- Não são tipos fundamentais: `#include <string>` e `#include <string_view>`

Container `std::string`

- Podemos criar, inicializar e atribuir valores a objetos do tipo `std::string` como outros objetos.
- `std::string` pode armazenar strings de diferentes comprimentos

```
#include <string>
int main(){
    std::string nome { "Leo" }; // inicializa nome com o literal "Leo"
    nome = "Leonardo";        // altera nome para "Leonardo"
    return 0;
}
```

- Strings podem ser caracteres numéricos

```
std::string myID{ "42" }; // "42" não é o mesmo do inteiro 42!
```

std::cout para Saída de Texto

- Objetos `std::string` podem produzir saídas como esperado utilizando `std::cout`

```
#include <iostream>
#include <string>
int main(){
    std::string nome { "Leo" };
    std::string vazio { };
    std::cout << "Meu nome é: " << nome << '\n';
    std::cout << '[' << vazio << ']';
    return 0;
}
```

std::cin para Entrada de Texto

- Usar std::string com std::cin pode trazer algumas surpresas.

```
#include <iostream>
#include <string>
int main(){
    std::cout << "Digite seu nome completo: ";
    std::string nome{};
    std::cin >> nome; // não funciona como esperado pois std::cin quebra com espaço
    std::cout << "Digite sua cor favorita: ";
    std::string cor{};
    std::cin >> cor;
    std::cout << "Seu nome é " << nome << " e sua cor favorita é " << cor << '\n';
    return 0;
}
```

std::getline() para Entrada de Texto

- Para ler uma linha completa de entrada numa string, é melhor usar a função `std::getline()`.
- `std::getline()` requer dois argumentos: o primeiro é `std::cin`, e o segundo é a sua variável string.

```
#include <iostream>
#include <string> // Para std::string e std::getline
int main(){
    std::cout << "Digite seu nome completo: ";
    std::string nome{};
    std::getline(std::cin >> std::ws, nome); // para ler uma linha completa de texto
    std::cout << "Digite sua cor favorita: ";
    std::string cor{};
    std::getline(std::cin >> std::ws, cor); // para ler uma linha completa de texto
    std::cout << "Seu nome é " << nome << " e sua cor favorita é " << cor << '\n';
    return 0;
}
```

- O manipulador de entrada `std::ws` diz ao `std::cin` para ignorar qualquer espaço em branco à esquerda antes da extração

Comprimento de um `std::string`

- Se quisermos saber quantos caracteres tem uma `std::string`, podemos perguntar pelo seu comprimento.

```
#include <iostream>
#include <string>
int main(){
    std::string nome{ "Leo" };
    std::cout << nome << " possui " << nome.size() << " caracteres\n"; //C++11
    std::cout << name << " possui " << std::ssize(name) << " caracteres\n"; //C++20
    return 0;
}
```


Literais `std::string`

- Os literais de strings entre aspas duplas (como "Olá, mundo!") são strings de estilo C.
- Podemos criar literais de string com o tipo `std::string` usando um sufixo `s` após o literal de string entre aspas duplas.
- O `s` deve ser minúsculo.

```
#include <iostream>
#include <string> // para std::string
int main(){
    using namespace std::string_literals; // acesso ao sufixo s
    std::cout << "foo\n";    // sem sufixo s é um literal estilo C
    std::cout << "goo\n"s;   // com sufixo s é um literal std::string
    return 0;
}
```

- Usar literais `std::string` tornam o uso de strings mais fáceis.

Ineficiência na Cópia de Strings

- Na inicialização, assim como nas passagens de parâmetro por valor, literais são copiados para a memória
- Para tipos fundamentais, é uma operação rápida, mas para strings o custo deve ser considerado.

```
#include <iostream>
#include <string>

void printString(std::string str){ // str faz uma cópia da sua inicialização
    std::cout << str << '\n';
}

int main(){
    std::string s{ "Hello, world!" }; // s faz uma cópia da sua inicialização
    printString(s);
    return 0;
}
```

std::string_view

- O C++17 introduziu o `std::string_view` para proporcionar acesso (somente leitura) sem cópia a uma string.

```
#include <iostream>
#include <string_view> // C++17
// str proporciona acesso somente leitura para qualquer argumento passado
void printSV(std::string_view str){ // agora uma std::string_view
    std::cout << str << '\n';
}
int main(){
    std::string_view s{ "Hello, world!" }; // agora uma std::string_view
    printSV(s);
    return 0;
}
```

Inicialização do `std::string_view`

- Um objeto `std::string_view` pode ser inicializado com uma string estilo C, uma `std::string`, ou outra `std::string_view`.

```
#include <iostream>
#include <string>
#include <string_view>

int main(){
    std::string_view s1 { "Hello, world!" }; // inicializado com string estilo C
    std::cout << s1 << '\n';
    std::string s{ "Hello, world!" };
    std::string_view s2 { s }; // inicializado com std::string
    std::cout << s2 << '\n';
    std::string_view s3 { s2 }; // inicializado com std::string_view
    std::cout << s3 << '\n';
    return 0;
}
```

Parâmetros `std::string_view`

- Um parâmetro `std::string_view` pode receber argumentos com uma string estilo C, uma `std::string`, ou outra `std::string_view`.
- Tanto uma string estilo C como uma `std::string` serão implicitamente convertidas para uma `std::string_view`.

```
#include <iostream>
#include <string>
#include <string_view>

void printSV(std::string_view str){
    std::cout << str << '\n';
}

int main(){
    printSV("Hello, world!"); // chamada com string estilo C
    std::string s2{ "Hello, world!" };
    printSV(s2); // chamada com std::string
    std::string_view s3 { s2 };
    printSV(s3); // chamada com std::string_view
    return 0;
}
```

Conversão de `std::string_view` para `std::string`

- `std::string_view` não converte implicitamente para `std::string`.
- Para converter uma `std::string_view` existente para uma `std::string` use `static_cast`.

```
static_cast<std::string>(sv)
```

Atribuição para `std::string_view`

- Atribuir uma nova string a uma `std::string_view` faz com que a `std::string_view` exiba a nova string.

```
#include <iostream>
#include <string>
#include <string_view>
int main(){
    std::string nome { "Leonardo" };
    std::string_view sv { nome }; // sv vizualiza nome
    std::cout << sv << '\n'; // imprime Leonardo
    sv = "Leo"; // sv vizualiza "Leo" (não altera nome)
    std::cout << sv << '\n'; // imprime Leo
    std::cout << nome << '\n'; // imprime Leonardo
    return 0;
}
```

Literais `std::string_view`

- Os literais de strings entre aspas duplas (como "Olá, mundo!") são strings de estilo C.
- Podemos criar literais de string com o tipo `std::string_view` usando um sufixo `sv` após o literal de string entre aspas duplas.
- O `sv` deve ser minúsculo.

```
#include <iostream>
#include <string>      // para std::string
#include <string_view> // para std::string_view
int main(){
    using namespace std::string_literals;      // acesso ao sufixo s
    using namespace std::string_view_literals; // acesso ao sufixo sv
    std::cout << "foo\n";    // sem sufixo s é um literal estilo C
    std::cout << "goo\n"s;   // com sufixo s é um literal std::string
    std::cout << "moo\n"sv;  // com sufixo sv é um literal std::string_view
    return 0;
}
```


constexpr std::string_view

- std::string_view tem suporte para constexpr

```
#include <iostream>
#include <string_view>
int main(){
    constexpr std::string_view s{ "Hello, world!" }; // s é uma string simbólica const
    std::cout << s << '\n'; // s será substituído por "Hello, world!" em tempo de compilação
    return 0;
}
```

Sumário, Atividades e Dúvidas

- Arranjos Estilo C, `std::array` e `std::vector`
- `std::string` e `std::string_view`
- Atividade 1: Escreva os seguintes programas em C++.
 - Defina um arranjo estilo C para armazenar os 6 primeiros números ímpares e exibir os elementos.
 - Defina um arranjo `std::array` para armazenar os 6 primeiros números ímpares e exibir os elementos.
 - Defina um arranjo `std::vector` para armazenar os 6 primeiros números ímpares e exibir os elementos.
- Atividade 2: Escreva um programa em C++ que preencha e exiba as seguintes informações de um estudante usando `std::string`:
 - Nome Completo
 - Curso
 - Duas unidades curriculares matriculadas

Sumário, Atividades e Dúvidas (II)

- Atividade 3: Escreva o mesmo programa C++ da Atividade 2 usando `std::string_view`
- Perguntas?
- Anotem e tragam as dúvidas na próxima aula

Referências

- Learn C++. <https://www.learncpp.com/>

Obrigado!